Getting Ready for the Stack Simulator

Tech Stack and Tools: HTML, CSS, Bootstrap, JQuery, Javascript, Git,

Github, and VS Code *Reading activity based on the starter template*

Name: Adrian Sibayan Section: BSIT WMT 2-2 Date: 10/26/2027

Read and answer the five guide questions. Focus on what each part does. You do not need to memorize syntax, edit the files, or run the program.

A familiar stack with a new display

You have already used JavaScript stacks in the console. A stack follows Last In, First Out, or LIFO. When we push 100, 200, 300, and 400 in that order, 400 becomes the top. Peek reads the top without removing it. Pop removes and returns the top.

Our starter uses the same Stack class with a web page. It adds four values automatically when the document is ready. The Push, Pop, Peek, and Clear buttons are visible, but the starter has no click handlers to make them operate the stack. Connecting those controls will be part of the demonstration.

The role of each technology

HTML defines the page structure. The HTML starter includes a heading, a text input, four buttons, and areas labeled Operations, Stack Data, and Deleted from Stack. Elements such as div, input, and button describe these parts.

CSS defines appearance. The custom stylesheet sets the page background to #495579 and gives .push and .pop blocks a width and height of 50px. px means pixels. The .popped rule arranges the deleted area using a flex layout.

Bootstrap supplies ready-made CSS classes. btn and btn-success style the Push button. row and col help arrange the page. The starter links Bootstrap and jQuery from online sources, so those resources need to load for their features to work.

JavaScript manages behavior and data. The JavaScript starter defines the Stack class, including push(), pop(), peek(), and clear(). const stack = new Stack() creates an initially empty stack object.

jQuery is a JavaScript library. In this starter, $(document).ready(...) runs a function when the document is ready. The function selects #stack and uses .prepend() to add visible blocks. It does not yet read the input or respond to the operation buttons.

An element and its classes

```
<button class="btn btn-success" type="button">Push</button>
```

button is the element name. class and type are attributes. Push is the visible label. This starter button has CSS classes but no id attribute. A button label describes a control; JavaScript must still connect that control to an action.

Selectors and the DOM

A selector chooses elements. #stack selects the element with id="stack". .push selects elements with class="push". An id should uniquely identify one element, while several elements can share a class. The DOM, or Document Object Model, is the browser's representation of the page that JavaScript can change.

How the Starter Displays the Stack

Reading one statement

```
$("#stack").prepend(
 `<div class="row bg-primary push">${stack.push(100)[0]}</div>` );
```

This is the first insertion statement, wrapped onto shorter lines. stack.push(100) stores 100 and returns ["100", 0]. [0] selects the first item in that returned array: the text "100". Backticks create a template literal, and ${...} inserts the result into the HTML text.

$("#stack") selects the display container. .prepend(...) inserts the new div before its existing children. row and bg-primary are Bootstrap classes; push is a class styled in the custom CSS. Changing the Stack object and adding its visible block are separate actions in this statement.

Four statements run when the document is ready

The starter repeats the insertion for 100, 200, 300, and 400, in that order. Each new block goes before the previous blocks. Therefore, the display reads 400, 300, 200, 100 from top to bottom. The logical stack size is 4, and the top is 400. Refreshing creates a fresh object and repeats these four insertions.

The 500 in Deleted from Stack is already written in the HTML. It is a static sample block. No pop() call placed it there, and it is not one of the four values stored by the initialization code.

Guide questions

1. Match each job to HTML, CSS, Bootstrap, JavaScript Stack class, or jQuery. Use each answer once.
   a. Defines the heading, input, and buttons: **HTML**
   b. Sets the custom block width to 50px: **CSS**
   c. Supplies btn-success and bg-primary: **Bootstrap**
   d. Stores the values and implements push(): **JavaScript Stack class**
   e. Selects #stack and inserts visible blocks: **jQuery**

2. What does #stack select? Does the starter Push button have an id?
   The element with `id="stack"` (the display container). No, the starter Push button has CSS classes but no `id` attribute.

3. After all four insertion statements run, write the displayed values from top to bottom and give the stack size. Which value would Pop remove first?

   The displayed values from top to bottom are 400, 300, 200, 100 (because each new item is prepended before the previous ones). The stack size is 4, and 400 would be removed first by Pop following Last In, First Out (LIFO) rules.

4. Does the visible 500 prove that the program has performed Pop? Explain.

   No. The visible 500 in "Deleted from Stack" is a static sample block already written directly into the HTML; it was not placed there by a `pop()` call.

5. Write one question about connecting the input or buttons to the stack that you want answered during the demonstration.

How do we write a jQuery click handler for the Push button that reads the value from the text input field, adds it to the stack object, and updates the web display?